LENDING CLUB · CREDIT SCORING

Predicting Loan Default

Using only data known when the loan is approved

DATASET

Lending Club accepted loans (US peer-to-peer lending)

2007 – 2018 issue period
2,260,701 rows (loans)
151 columns

GOAL

Predict if a new loan will default

→ Output: probability of default (0 – 1)
→ Label: Charged Off $= 1$, Fully Paid $= 0$
→ Input: only data known at approval

1 Data Dictionary · 2 Target · 3 Split · 4 Cleaning · 5 Selection · 6 Engineering · 7 Modeling · 8 Test Results · + 3 Experiments

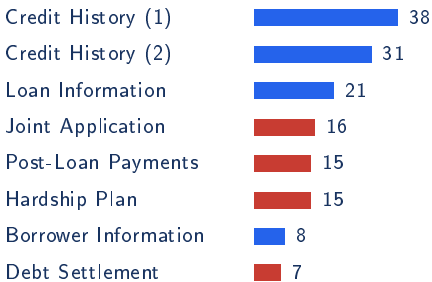
[Student 1], [Student 2] | Statistical Learning (DS66A / DS66B) | Supervisor: [Name]

STEP 1 · NOTEBOOK 01

Data Understanding: Data Dictionary

151 columns in 8 groups, each tagged: known at application?

COLUMNS PER GROUP



Red = groups only filled after the loan starts

TIMING TAG

103 Yes: known at application

41 No: after loan or ID

7 Uncertain: timing not clear

151 / 151

dictionary matches the raw file header

STEP 2 · NOTEBOOK 01

Data Ingestion & Target Definition

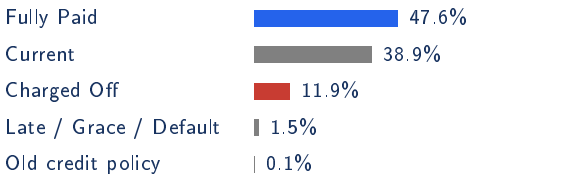
Only loans with a final result get a label

2,260,701
rows × 151 columns

1,345,310
labelled rows

20.0%
default rate (labelled)

LOAN STATUS (% OF ALL ROWS)



label 0 = Fully Paid · label 1 = Charged Off · gray = removed (no final result)

LOAN TERM (MATURED, FROM 2012)



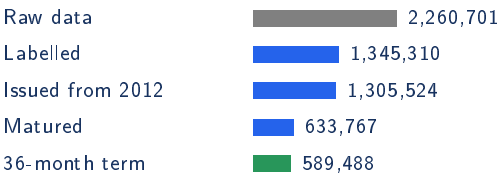
Keep 36-month loans only, so all 3 sets share one term.

STEP 3 · NOTEBOOK 01

Data Filtering & Out-of-Time Split

Train on old loans, test on newer loans: 70 / 15 / 15 by date

FILTER FUNNEL (ROWS)



THREE SETS

Train	408,188 69.2%	2012-01 → 2015-05 default rate 13.7%
Val	92,631	2015-06 → 09 default 14.7%
Test	88,669	2015-10 → 12 default 14.8%

STEP 4 · NOTEBOOK 02

Data Cleaning: 6 Logic Rules

Each rule checks one row; the same rule runs on train, val and test

#	Logic rule (check)	Why	Handling
1	`annual_inc` ≤ 0	Income must be positive	Set to missing
2	`loan_amnt` ≤ 0	Loan amount must be positive	Set to missing
3	`dti` < 0 or > 100	Debt-to-income must be 0–100	Set to missing
4	`revol_util` > 100	Over 100% is possible; extremes are noise	Cap at 102 (train p99.9)
5	`fico_low` $>$ `fico_high`	Low bound must not pass high bound	Set both to missing
6	`earliest_cr_line` $>$ `issue_d`	First credit must be before the loan	Set to missing

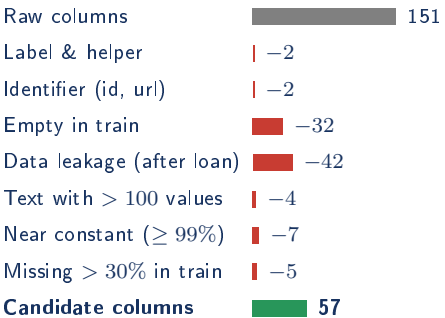
Anti-leakage rules

Split first, transform later · median, mean, std, caps learned on **train only** · pipeline `fit` on train, val/test only `transform`

STEP 5 · NOTEBOOK 02

Feature Selection: 151 → 57

Each column goes through filters in a fixed order; first match removes it



Leakage check

Single-column ROC-AUC on train. Best legitimate column: `sub_grade` **0.6565**.

Post-loan columns are far stronger:

`last_fico_range_low` 0.9074

`recoveries` 0.9054

`total_rec_prncp` 0.8280

STEP 6 · NOTEBOOK 02

Feature Engineering & Preprocessing: 57 → 153

New features, remove duplicates, then impute, scale and one-hot in one pipeline

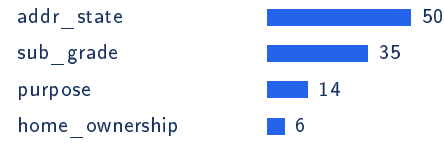
57 candidates → **+1** issue_date (anchor) → **+6** new features → **−14** duplicates & high corr. → **50** 45 num + 5 cat →

153 final matrix

6 NEW FEATURES

`fico_avg`: mean of FICO range
`credit_history_years`: years of credit
`loan_to_income`: loan / income
`log_annual_inc`, `log_revol_bal`: fix skew
`emp_length_years`: text → number

ONE-HOT COLUMNS (108 TOTAL)



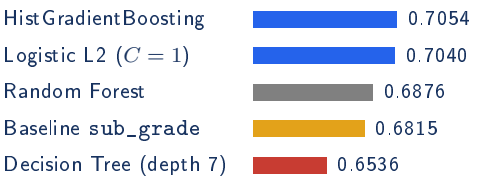
45 numeric: median impute + StandardScaler

STEP 7 · NOTEBOOK 03

Model Training & Validation

5 models tuned with TimeSeriesSplit; compared on the validation set

VALIDATION ROC-AUC (axis starts at 0.60)



OVERFITTING (TRAIN − VAL AUC)

Random Forest $+0.018$ · HistGB $+0.004$ · Logistic -0.017

BOOTSTRAP 95% CI · ΔAUC VS BASELINE

HistGB [+0.021, +0.027]
Logistic L2 [+0.020, +0.025]
Random Forest [+0.002, +0.010]
Decision Tree [−0.033, −0.023]

STEP 8 · NOTEBOOK 03

Out-of-Time Test Results & Conclusion

Test set used once: 88,669 loans, 2015-10 to 2015-12

	ROC-AUC	Gini	KS
Baseline sub_grade	0.6818	36.4%	26.5%
Logistic L2 CHOSEN	**0.7039**	40.8%	29.9%
HistGradientBoosting	0.7055	41.1%	30.0%

HISTGB − LOGISTIC (95% CI)

$[-0.0002,\ +0.0031]$ → contains 0: no real difference

Decision: Logistic L2

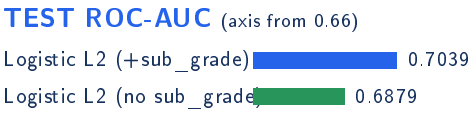
- ✓ Beats baseline: $+0.022$ AUC
- ✓ Same level as HistGB
- ✓ No overfitting, fast to train
- ✓ Each coefficient is easy to explain

EXPERIMENT 1 · EXPERIENCES FOLDER

Ablation Study: Model without sub_grade

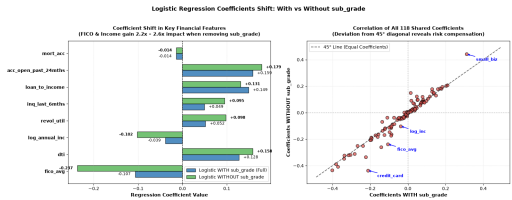
Remove Lending Club's own grade: 153 → 118 features. Is the model still better?

TEST ROC-AUC (axis from 0.66)



+0.0075 no sub_grade vs baseline
95% CI [+0.003, +0.012]

+0.0150 value added by sub_grade
95% CI [+0.013, +0.017]



sub_grade absorbs the risk signal; without it, weight moves back to FICO, income and credit use ($1.2\times$ – $2.6\times$).

EXPERIMENT 2 · EXPERIENCES FOLDER

Compact Model & Credit Scorecard

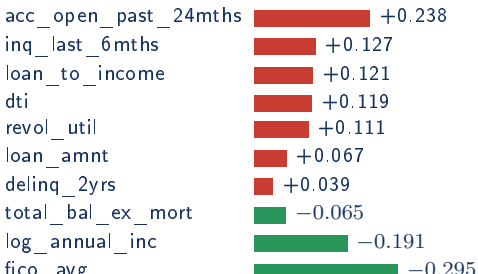
12 core features, no `sub_grade`, turned into a 300 – 850 style score

153 → 30
columns (−80%)

0.673
test ROC-AUC (Gini 34.6%)

494 – 757
score range (validation)

WEIGHTS OF 10 NUMERIC FEATURES



SCORE SCALING

Base score: 600 at odds 5 : 1
PDO: 20 points
Example cut-off: 615
Higher score = lower risk

2 CATEGORICAL FEATURES

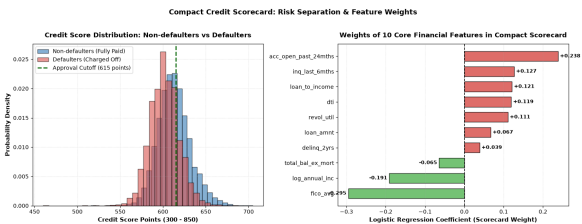
`home_ownership`, `purpose`

Fewer inputs: cheaper data collection, easy to explain to a loan officer.

EXPERIMENT 3 · COMPACT SCORECARD ON TEST

Credit Score Distribution: Good vs Bad Borrowers

Compact 12-feature scorecard · test set, 88,669 loans



DEFAULT RATE BY BAND (average 14.8%)

< 590 Reject	30.2%	
590–614 Review	17.3%	
615–639 Approve	8.7%	
≥ 640 Approve	3.3%	

9× default gap: lowest vs highest band

25.1% test KS

BACKUP

Evaluation metrics

ROC-AUC: chance a bad loan scores above a good loan

Gini $= 2\times$ AUC -1: standard score in credit risk

KS: max gap between bad and good cumulative rates, $\max(\mathrm{TPR} - \mathrm{FPR})$

No probability cut-off is needed: all three measure ranking quality. The same function is used for every model and data set.

NB03 cell 5

```
def calculate_metrics(y_true, proba):
  auc = roc_auc_score(y_true, proba)
  gini = 2 * auc - 1
  fpr, tpr, _ = roc_curve(y_true, proba)
  ks = np.max(tpr - fpr)
  return auc, gini, ks
```

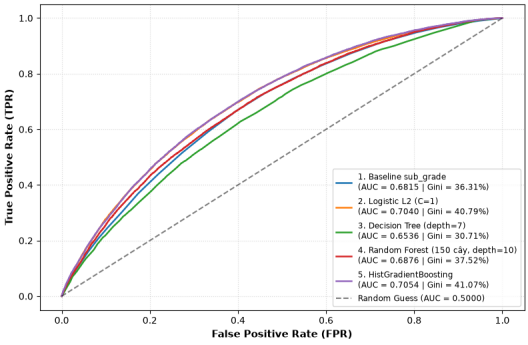
BACKUP

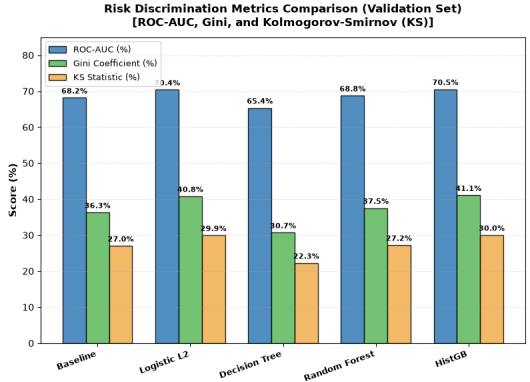
Hyperparameter tuning and train vs validation

Model	Grid	Best	CV AUC
Logistic L2	$C \in \{0.001 \dots 10\}$	$C = 1.0$	0.6855
Decision Tree	max_depth 3–10	depth $= 7$	0.6355
Random Forest	trees 50/100/150, depth 6/8/10, leaf 30/50/60	150, 10, 50	0.6741
HistGradientBoosting	lr 0.05–0.20, depth 4–10, iter 50–150	0.05, 10, 150	0.6852

Model (val = 92,631 loans)	AUC train	AUC val	Gap	Gini val	KS val
Baseline `sub_grade`	0.6565	0.6815	-0.0250	36.31%	27.02%
Logistic L2 ($C = 1$)	0.6873	0.7040	-0.0167	40.79%	29.94%
Decision Tree (depth 7)	0.6541	0.6536	$+0.0006$	30.71%	22.25%
Random Forest (150, depth 10)	0.7060	0.6876	$+0.0184$	37.52%	27.21%
HistGradientBoosting	0.7096	0.7054	$+0.0042$	41.07%	30.02%

Validation curves and bootstrap





Bootstrap $B = 1{,}000$ resamples of the validation set; if the 95% CI of ΔAUC does not contain 0, the gain is real.

Model	ΔAUC vs baseline	95% CI
Logistic L2	$+0.0225$	$[+0.0196, +0.0253]$
Decision Tree	-0.0280	$[-0.0329, -0.0233]$
Random Forest	$+0.0061$	$[+0.0024, +0.0102]$
HistGradientBoosting	$+0.0238$	$[+0.0206, +0.0270]$

BACKUP

Known limits

- Model covers 36-month loans only; test has only 3 months of late 2015
- 32 columns are empty in train; 17 of them only start in val/test
- "Months since event" columns were removed ($> 30\%$ missing) though missing may mean "never happened"
- `issue_d` is used in place of application date; only accepted loans exist in the data
- RF and HistGB tuning grids hit the edge of the search space
- One-hot gives 108 sparse dummies: fits Logistic well, harder for trees
- Scorecard constants (600, PDO 20, odds 5:1) are conventions, not checked against an official standard

References I

C. Serrano-Cinca, B. Gutiérrez-Nieto, and L. López-Palacios, "Determinants of default in P2P lending," *PLoS ONE*, vol. 10, no. 10, p. e0139427, 2015.

M. Malekipirbazari and V. Aksakalli, "Risk assessment in social lending via random forests," *Expert Systems with Applications*, vol. 42, pp. 4621–4631, 2015.

H. Qian, B. Wang, P. Ma, L. Peng, S. Gao, and Y. Song, "Managing dataset shift by adversarial validation for credit scoring," arXiv preprint arXiv:2112.10078, 2021.

N. Siddiqi, *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards*, 2nd ed., ser. Wiley and SAS Business Series. Hoboken, NJ: John Wiley & Sons, 2017.

N. George, "Lending club loan data (accepted and rejected loans, 2007–2018)," Kaggle dataset (user `wordsforthewise`), https://www.kaggle.com/datasets/wordsforthewise/lending-club, accessed 2026.

D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: a review," *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

References II

- J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.
- B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.